

LibraryWeb

aplikacja do obsługi systemu bibliotecznego

Autor

Piotr Owczarek – 241232

1. Cel i zakres projektu

Aplikacja „LibraryWeb” jest systemem webowym służącym do zarządzania zasobami bibliotecznymi, wypożyczeniami, rezerwacjami, opłatami oraz kontami użytkowników.

Projekt obejmuje:

- zarządzanie książkami i ich egzemplarzami
- obsługę wypożyczeń i zwrotów
- automatyczne naliczanie opłat za zwrot książki po terminie
- rezerwacje książek
- panel administratora i bibliotekarza

2. Użyte technologie

Projekt został zrealizowany z użyciem technologii:

- backend: JAVA + springboot
- frontend: React + TypeScript + bootstrap
- baza danych: MySQL

3. Role w systemie:

- Administrator – zarządzanie użytkownikami, kategoriami, autorami, książkami, egzemplarzami, wypożyczeniami, opłatami
- Bibliotekarz – obsługa wypożyczeń, zwrotów, rezerwacji, zamykania opłat, dodawanie i edycja danych książki, autora, kategorii
- Użytkownik zalogowany – przeglądanie katalogu, rezerwacje, podgląd swoich wypożyczeń i opłat
- Użytkownik niezalogowany – przeglądanie katalogu, logowanie i rejestracja

4. Przypadki użycia:

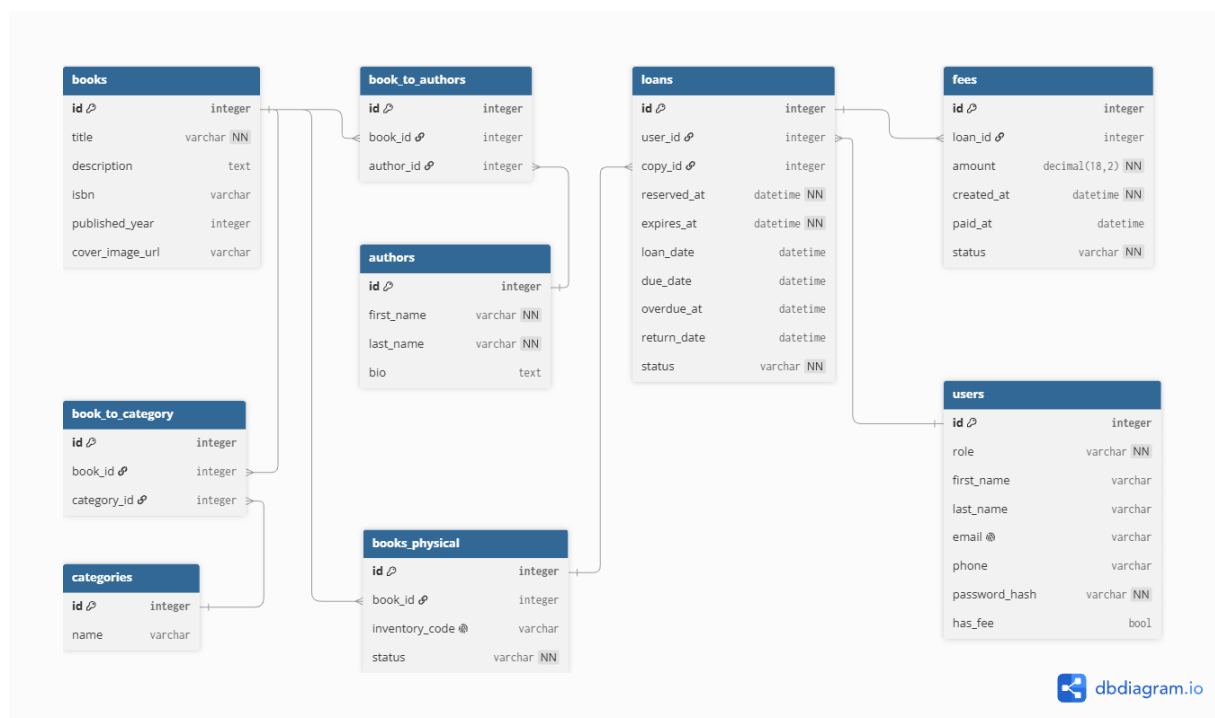
- Logowanie i Rejestracja(użytkownik niezalogowany)
- Przeglądanie katalogu(użytkownik niezalogowany i zalogowany)
- Rezerwacja książki(użytkownik zalogowany)
- Wypożyczenie egzemplarza(użytkownik zalogowany)
- Zwrot egzemplarza(użytkownik zalogowany)
- Utworzenie opłaty za późne oddanie(bibliotekarz i administrator)
- Oznaczenie opłaty za późne oddanie jako zapłaconej(bibliotekarz i administrator)
- Zarządzanie użytkownikami(administrator)
- Zarządzanie książkami(bibliotekarz i administrator)
- Zarządzanie autorami(bibliotekarz i administrator)
- Zarządzanie kategoriami książek(bibliotekarz i administrator)

5. Opis dostępnych funkcjonalności

System umożliwia:

- Dodawanie i edycję książek
- Dodawanie egzemplarzy
- Wypożyczanie i zwroty
- Rezerwacje książek
- Zarządzanie kontami użytkowników
- Podgląd historii wypożyczeń

6. Diagram bazy danych



Baza składa się z tabeli:

- **books** – zawiera informacje o książce, takie jak: tytuł, opis, rok wydania, link do okładki książki, odniesienie do tabeli autorów
- **authors** – zawiera informacje o autorach, takie jak: imię, nazwisko, biografię
- **book_to_authors** – tabela tworzy powiązanie many-to-many, które pozwala każdej książce mieć wielu autorów
- **categories** – zawiera informacje o nazwach kategorii książek

- **book_to_category** – tabela tworzy powiązanie many-to-many, które pozwala każdej książce mieć wiele różnych kategorii
- **books_physical** – zawiera informacje o fizycznych egzemplarzach, takie jak: kod inwentarza, status egzemplarza(dostępny, zarezerwowany, wypożyczony), odniesienie do informacji o książce w tabeli **books**
- **users** – zawiera informacje o kontach użytkowników, takie jak: imię, nazwisko, rola(wartość: administrator, bibliotekarz lub użytkownik), email, numer telefonu, hash hasła, pole przechowujące wartość czy użytkownik ma niezapłacone opłaty
- **loans** – zawiera informacje o wypożyczeniach i rezerwacjach użytkowników, takie jak: datę rezerwacji(dodania do koszyka), datę wygaśnięcia rezerwacji z koszyka, datę wypożyczenia, datę końca okna zwrotu bez opłat, datę wystawienia opłaty za przekroczenie terminu zwrotu, datę zwrotu, odniesienie do użytkownika który wypożyczył egzemplarz w tabeli **users**, odniesienie do danych wypożyczonego egzemplarza w tabeli **books_physical**, pole przechowujące obecny status wypożyczenia(wartość: zarezerwowany, aktywny, zaległy, zwrócony)
- **fees** – zawiera informację o opłatach za przekroczenie terminu oddania, takie jak: odniesienie do danych wypożyczenia, kwotę kary/opłaty, datę utworzenia, datę zapłacenia, status opłaty(wartość: oczekująca, zapłacona), odniesienie do danych wypożyczenia w tabeli **loans**

7. Opis relacji i ich znaczenia

Relacje w bazie danych zapewniają:

- powiązanie książek z autorami – relacja many-to-many przez tabelę **book_to_authors**, wiele książek może mieć wiele autorów i wiele autorów może mieć wiele książek
- powiązanie egzemplarzy z książkami – relacja many-to-one, wiele książek może mieć informacje o tej samej książce
- powiązanie użytkownika z wypożyczeniami – relacja one-to-many, jeden użytkownik może mieć wiele wypożyczeń, ale każde wypożyczenie należy do dokładnie jednego użytkownika
- powiązanie wypożyczenia z egzemplarzem – relacja one-to-many, jedno wypożyczenie dotyczy jednego egzemplarza, ale jeden egzemplarz może być wypożyczony wiele razy
- powiązanie wypożyczenia z opłatą – relacja one-to-one, każde wypożyczenie może mieć tylko jedną opłatę, każda opłata odnosi się tylko do jednego wypożyczenia

8. Backend

8.1. Opis wybranego komponentu: SecurityConfig

```
@Configuration @EnableWebSecurity
public class SecurityConfig {

    @Value("${app.jwt.secret}")
    private String jwtSecret;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf((CsrfConfigurer<HttpSecurity> csrf -> csrf.disable()))
            .cors((CorsConfigurer<HttpSecurity> cors -> {}))
            .sessionManagement((SessionManagementConfigurer<HttpSecurity> session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests((AuthorizationManagerRequestMatcher<...> auth -> auth
                .requestMatchers(HttpMethod.GET, ...patterns: "/books", "/books/").permitAll()
                .requestMatchers(HttpMethod.GET, ...patterns: "/categories", "/categories/", "/categories/counts").permitAll()
                .requestMatchers(...patterns: "/user/login", "/user/register", "/error", "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer((OAuth2ResourceServerConfigurer<HttpSecurity> oauth2 -> oauth2
                .jwt((JwtConfigurer jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
            );

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    public JwtDecoder jwtDecoder() {
        return NimbusJwtDecoder.withSecretKey(jwtSecretKey()).macAlgorithm(MacAlgorithm.HS256).build();
    }

    @Bean
    public JwtEncoder jwtEncoder() { return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecretKey())); }

    @Bean
    public JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter((Jwt jwt -> {
            String role = jwt.getClaimAsString("role");
            if (role == null || role.isBlank()) {
                return Collections.emptyList();
            }

            Collection<GrantedAuthority> authorities = List.of(new SimpleGrantedAuthority("ROLE_" + role));
            return authorities;
        }));
        return converter;
    }

    private SecretKey jwtSecretKey() {
        return new SecretKeySpec(jwtSecret.getBytes(StandardCharsets.UTF_8), "HmacSHA256");
    }
}
```

Komponent SecurityConfig jest centralnym komponentem odpowiedzialnym za konfigurację bezpieczeństwa aplikacji. Definiuje on:

- sposób uwierzytelniania użytkowników za pomocą JWT (każde zapytanie na stronach użytkownika wymaga dołączenia tokena JWT)
- zasady autoryzacji dla poszczególnych endpointów
- konfigurację sesji (stateless – backend nie przechowuje informacji o stanie ani historii interakcji użytkownika pomiędzy kolejnymi rządzeniami)
- integrację z modułem OAuth2 Resource Server

- sposób dekodowania i walidacji tokenów
- mapowanie roli użytkownika z tokena na uprawnienia Spring Security
- definiowanie które zasoby/endpointy API są publicznie dostępne

8.2. Zabezpieczenie

Zabezpieczenie backendu zostało oparte na mechanizmie JSON Web Token (JWT), który jest obecnie standardem w aplikacjach typu SPA + REST API.

Każdy użytkownik (czytelnik, bibliotekarz, administrator) po poprawnym zalogowaniu otrzymuje token JWT, który musi być dołączany do każdego kolejnego zapytania kierowanego do API.

W konfiguracji SecurityConfig określono, które endpointy są publiczne, a które wymagają tokena:

- - publiczne:
 - /user/login
 - /user/register
 - /books (GET)
 - /categories (GET)
 - Swagger UI
- wszystkie pozostałe endpointy:
 - wymagają poprawnego JWT
 - wymagają odpowiedniej roli (np. ADMIN, LIBRARIAN)

8.3. Endpointy API

W aplikacji stworzyłem następujące endpointy:

- User – dane o użytkownikach
 - GET /user/{id} — pobiera dane użytkownika po ID użytkownika
 - PUT /user/{id} — aktualizuje dane użytkownika

- DELETE /user/{id} — usuwa użytkownika
- PUT /user/changePassword/{id} — zmienia hasło użytkownika
- POST /user/register — rejestracja nowego użytkownika
- POST /user/login — logowanie użytkownika
- GET /user — pobiera listę wszystkich użytkowników

- Book Copy – dane o egzemplarzach

- GET /copies/{id} — pobiera dane o fizycznym egzemplarzu książki
- PUT /copies/{id} — aktualizuje egzemplarz
- DELETE /copies/{id} — usuwa egzemplarz
- GET /copies — pobiera listę wszystkich egzemplarzy
- POST /copies — tworzy nowy egzemplarz
- PATCH /copies/{id}/status — zmienia status egzemplarza (AVAILABLE/BORROWED/RESERVED)

- Category – dane o kategoriach

- GET /categories/{id} — pobiera dane o kategorii
- PUT /categories/{id} — aktualizuje kategorię
- DELETE /categories/{id} — usuwa kategorię
- GET /categories — lista wszystkich kategorii
- POST /categories — tworzy nową kategorię
- GET /categories/counts — zwraca kategorie z liczbą książek które są przypisane do kategorii

- Book – dane o książkach

- GET /books/{id} — pobiera dane książki
- PUT /books/{id} — aktualizuje książkę
- DELETE /books/{id} — usuwa książkę
- GET /books — lista wszystkich książek
- POST /books — tworzy książkę
- GET /books/available — zwraca dostępne książki (mają status AVAILABLE)

- Author – dane o autorach

- GET /authors/{id} — pobiera dane autora

- PUT /authors/{id} — aktualizuje autora
- DELETE /authors/{id} — usuwa autora
- GET /authors — lista wszystkich autorów
- POST /authors — tworzy autora

- Loan – dane o rezerwacjach

- POST /loans/return/{loanId} — oznacza zwrot książki
- POST /loans/reserve — rezerwuje egzemplarz, tzn. dodaje go do koszyka użytkownika
- POST /loans/overdue/{loanId} — oznacza wypożyczenie jako OVERDUE(przekroczenie terminu oddania bez opłaty)
- POST /loans/borrow/{loanId} — zmienia status wypożyczenia na wydany użytkownikowi
- GET /loans — lista wszystkich wypożyczeń/rezerwacji
- GET /loans/{id} — pobiera dane wypożyczeniu
- DELETE /loans/{id} — usuwa wypożyczenie
- GET /loans/userBooks/{userId} — pobiera wypożyczenia/rezerwacje książek użytkownika
- GET /loans/counts — statystyki wypożyczeń/rezerwacji dla użytkowników

- Fee

- GET /fees — lista wszystkich opłat
- POST /fees — tworzy nową opłatę
- POST /fees/markPaid/{id} — oznacza opłatę jako zapłaconą
- PATCH /fees/{id}/{status} — zmienia status opłaty (PAID/PENDING/CANCELLED)
- GET /fees/{id} — pobiera dane o opłacie
- DELETE /fees/{id} — usuwa opłatę
- GET /fees/userFees/{userId} — pobiera opłaty jednego użytkownika
- GET /fees/counts — statystyki opłat dla wszystkich użytkowników

9. Frontend

9.1. Komponenty

Główne komponenty użyte w aplikacji frontend:

- BooksMainPage – strona główna - wyświetla dostępne książki do wypożyczenia
- BookCardPage – karta książki - wyświetla informacje o książce
- AboutUsPage – strona About Us - wyświetla informacje o bibliotece
- CartPanel - Karta nakładająca się na obecnie otwartą stronę z listą książek dodanych do koszyka użytkownika
- LoginPage - strona logowania zarejestrowanego użytkownika
- RegisterPage - strona rejestracji użytkownika
- MyFeesPage - całą historię opłat zalogowanego użytkownika
- MySettingsPage – wyświetla dane użytkownika oraz formularz pozwalający użytkownikowi zmienić hasło konta.
- UserBooksPage – wyświetla historię wypożyczeń zalogowanego użytkownika
- AdminPanel – wyświetla panel admina/bibliotekarza w zależności od przypisanej roli zalogowanego użytkownika, zawiera podstrony które pozwalają na zarządzanie danymi biblioteki

9.2. Opis wybranego komponentu: AdminPanelPage

```
✓ const contentMap: Record<string, { title: string; description: string }> = {
✓   dashboard: {
✓     title: "Dashboard",
✓     description: "Manage library database, users, loans and settings from this panel.",
✓   },
✓   authors: {
✓     title: "Authors",
✓     description: "Manage authors and their bibliographies.",
✓   },
✓   books: {
✓     title: "Books",
✓     description: "Manage the catalog and add new items to the library.",
✓   },
✓   booksPhysical: {
✓     title: "Physical Books",
✓     description: "Manage the catalog and add new items to the library.",
✓   },
✓   categories: {
✓     title: "Categories",
✓     description: "Manage book categories",
✓   },
✓   users: {
✓     title: "Users",
✓     description: "Manage users, roles, and account statuses.",
✓   },
✓   loans: {
✓     title: "Loans",
✓     description: "Manage loans and their return deadlines.",
✓   },
✓   loanItems: {
✓     title: "Loan Items",
✓     description: "Manage individual loan items and their statuses.",
✓   },
✓   shoppingCarts: {
✓     title: "Shopping Carts",
✓     description: "Manage shopping carts and their contents.",
✓   },
✓   fees: {
✓     title: "Fees",
✓     description: "Manage fees and their payment statuses.",
✓   },
✓   feeItems: {
✓     title: "Fee Items",
✓     description: "Manage individual fee items and their statuses.",
✓   },
✓   settings: {
✓     title: "Settings",
✓     description: "System and admin panel preferences configuration.",
✓   },
✓ };
```

```

const sectionComponentMap: Record<string, JSX.Element> = {
  authors: <AuthorsDashboard />,
  books: <BooksDashboard />,
  booksPhysical: <BooksPhysicalDashboard />,
  categories: <CategoriesDashboard />,
  users: <UsersDashboard />,
  loans: <LoansDashboard />,
  loanItems: <LoanItemsDashboard />,
  fees: <FeesDashboard />,
  feeItems: <FeeItemsDashboard />,
};

const AdminPanelPage = () => {
  const { role, firstName, lastName, email } = useAuth();
  const navigate = useNavigate();
  const { section } = useParams();

  const activeSection = section ?? "dashboard";
  const currentContent = contentMap[activeSection] ?? contentMap.dashboard;

  useEffect(() => {
    if (role !== "ADMIN" && role !== "LIBRARIAN") {
      navigate("/");
    }
  }, [role]);

  const sectionContent = sectionComponentMap[activeSection] ?? (
    <div className="border rounded p-3 bg-light text-start">
      <h2 className="h5 mb-3">{currentContent.title}</h2>
      <p className="mb-0 text-muted">{currentContent.description}</p>
    </div>
  );

  return (
    <div className="container-fluid">
      <h1>{role} Panel</h1>
      <h2 className="mb-4">
        User: {firstName} {lastName} ({email})
      </h2>

      <div className="row g-4 align-items-start">
        <aside className="col-lg-3 col-md-4 col-12">
          <AdminNavPanel
            activeItem={activeSection}
            onSelect={item => navigate(`/admin-panel/${item}`)}
          />
        </aside>

        <div className="container col-lg-9 col-md-8 col-12 border">{sectionContent}</div>
      </div>
    </div>
  );
};

```

Komponent AdminPanelPage jest stroną panelu administracyjnego i bibliotecznego.

Pobiera dane o zalogowanym użytkowniku i jeśli nie jest on administratorem ani bibliotekarzem, zostaje on przekierowany na stronę główną.

W tym komponencie jest tworzona i wyświetlana lista nawigacji podstron której zawartość po wybraniu przez użytkownika zostanie wyświetlona po prawo do listy.

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

9.3. Tablica routingu

Aplikacja posiada następujące strony wraz z ich adresami/routingiem

- Home - /

LibraryWeb

Home

About us

My Books

My Fees

Shopping Cart

Profile

Search Books

Search Book by title



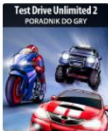
Test Drive Unlimited
Jacek Halas
Poradnik do gry „Test Drive Unlimited”
podzielony jest na tr...

Details



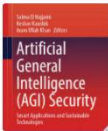
The Odessa Chronicles
Odessa is a Barn Owl with an attitude, who
befriended Jaxon ...

Details



Test Drive Unlimited 2
Jacek Halas, Maciej Stepińskiowski
W poradniku znajdziesz szczegółowy opis
przejęcia wszystkich...

Details



Artificial General Intelligence (AGI) Security
This book highlights a collection of state-of-
the-art resear...

Details

-

1

+

- About Us - /about-us

About Us page content

- My Books - /my-books

Your Books

Search loan by book title

Clear filter

#	Book Title	Author	Inventory Code	Reserved At	Expires At	Loaned At	Returned At	Overdue At	Status	All	Action
1	Test Drive Unlimited	Jacek Hales	333	14.09.2026, 02:42:51	21.09.2026, 23:59:59	14.09.2026, 02:51:00	-	-	ACTIVE		Details
2	The Odessa Chronicles		3	14.09.2026, 02:49:55	21.09.2026, 23:59:59	14.09.2026, 02:51:01	-	14.09.2026, 02:51:02	OVERDUE		Details

- My Fees - /my-fees

Your Fees

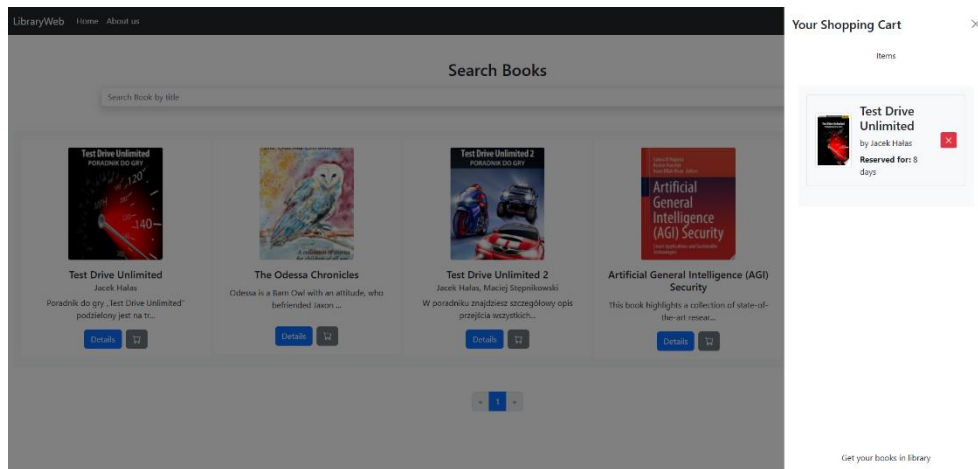
You have unpaid fees
Fees can be paid at the library counter.

Search fee by user, fee ID, loan ID or book title

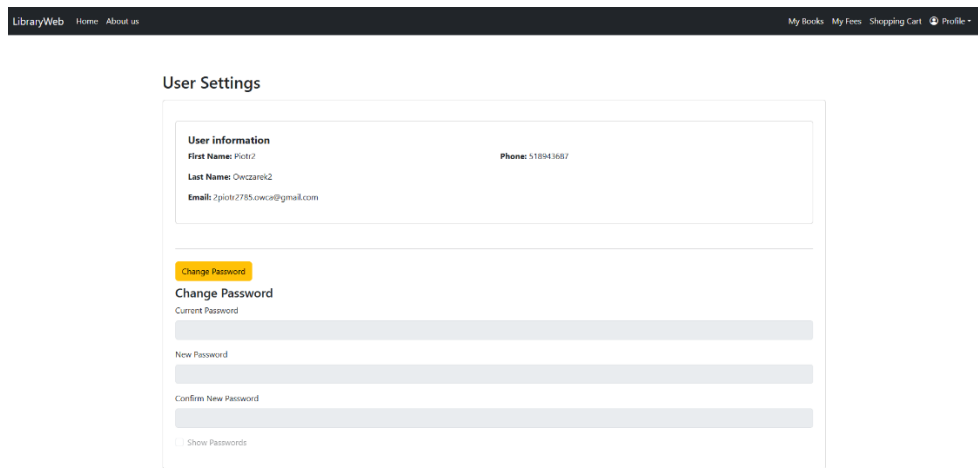
Clear filter

#	Book / Authors / Inventory Code	Created At	Amount(PLN)	Paid At	Status	All	Action
1	The Odessa Chronicles / 0 / 3	14.09.2026, 02:51:02	5.00 zł	-	PENDING		Details

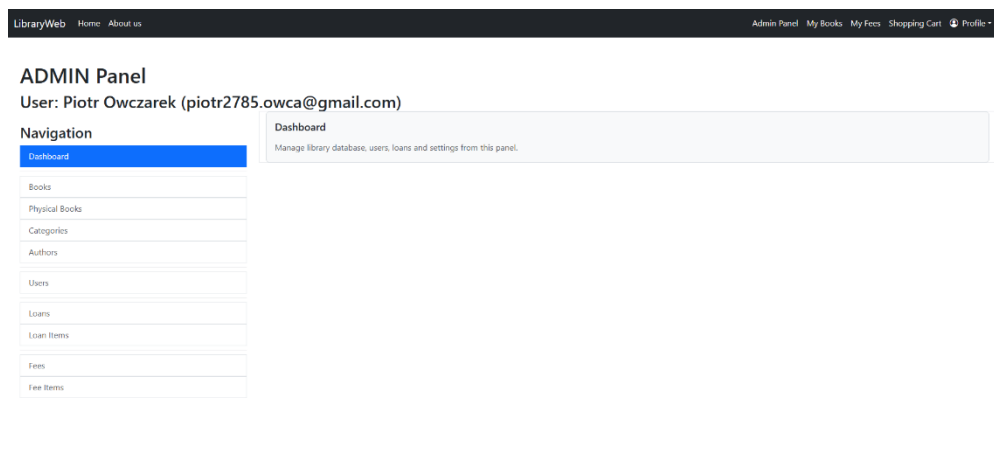
- Shopping Cart – bez routingu, jest częścią głównego paska nawigacyjnego



- Profile(Settings) - /settings



- Admin Panel - /admin-panel



- Librarian Panel - /librarian-panel

LibraryWeb

Home

About us

Librarian Panel

My Books

My Fees

Shopping Cart

Profile

LIBRARIAN Panel

User: Piotr Owczarek3 (3piotr2785.owca@gmail.com)

Navigation

Dashboard

Books

Physical Books

Categories

Authors

Users

Loans

Loan Items

Fees

Fee Items

Dashboard

Manage library database, users, loans and settings from this panel.

- Podstrony dla paneli Admin i Librarian:

- Books - /admin-panel/books

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard

Books

Physical Books

Categories

Authors

Users

Loans

Loan Items

Fees

Fee Items

Books Dashboard

Search book by title

Add Book

Clear filters

#	(ID) Title	Authors	ISBN	Published Year	Categories	Actions
1	(1) Test Drive Unlimited	Jacek Halas	9785042742620	2020	test	View Edit Delete
2	(2) The Odessa Chronicles	-	9781525520396	2018	-	View Edit Delete
3	(3) Test Drive Unlimited 2	Jacek Halas, Maciej Stepnikowski	9785042743054	2020	test	View Edit Delete
4	(4) Artificial General Intelligence (AGI) Security	-	9789819732227	2024	-	View Edit Delete

- Physical Books - /booksPhysical

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard

Books

Physical Books

Categories

Authors

Users

Loans

Loan Items

Fees

Fee Items

Physical Books Dashboard

Search physical book by inventory code or title

Add Physical Book

Clear filters

#	(ID) Inventory Code	(ID) Title (Authors)	Status	Actions
1	(2) 333	(1) Test Drive Unlimited (Jacek Halas)	BORROWED	View Edit Delete
2	(3) 4	(1) Test Drive Unlimited (Jacek Halas)	AVAILABLE	View Edit Delete
3	(4) 3	(2) The Odessa Chronicles ()	BORROWED	View Edit Delete
4	(7) 1	(3) Test Drive Unlimited 2 (Jacek Halas, Maciej Stepnikowski)	AVAILABLE	View Edit Delete
5	(8) 34	(4) Artificial General Intelligence (AGI) Security ()	AVAILABLE	View Edit Delete

- Categories - /admin-panel/categories

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Categories Dashboard

Search category by name or ID					Add Category	Clear filters
#	Category ID	Name	Number of books	Actions		
1	3	test	2	View	Edit	Delete

- Authors - /admin-panel/authors

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Authors Dashboard

Search author by name or biography					Add Author	Clear filters
#	Author ID	First Name	Last Name	Biography	Actions	
1	1	Jacek	Halas	-	View	Edit Delete
2	2	Maciej	Stepnikowski	f	View	Edit Delete

- Users - /admin-panel/users

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Users Dashboard

Search user

Add UserClear filters

#	User ID	First Name	Last Name	Email	Phone	Role	All	Has Unpaid Fees	All	Actions
1	1	Piotr	Owczarek	piotr2785.owca@gm...	518943687	ADMIN		No		View Edit Delete
2	2	Piotr2	Owczarek2	2piotr2785.owca@g...	518943687	USER		Yes		View Edit Delete
3	3	Piotr3	Owczarek3	3piotr2785.owca@g...	518943687	LIBRARIAN		No		View Edit Delete

- Loans - /admin-panel/loans

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Loans Report

Search loan by user, book title, author or inventory code

Clear filters

#	(ID) User	Total Loans	Reserved	Borrowed	Returned	Overdue	Actions
1	(2) Piotr2 Owczarek2	2	0	1	0	1	View Details
2	(1) Piotr Owczarek	1	0	0	1	0	View Details
3	(3) Piotr3 Owczarek3	0	0	0	0	0	View Details

- Loan Items - /admin-panel/loanItems

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Loans Dashboard

Search loan by book title

Clear filters

#	ID / User	Book	Reserved At	Expires At	Loaned At	Due At	Returned At	Overdue Set At	Status	Actions
1	1 / Piotr Owczarek	Test Drive Unlimited	11.09.2026, 22:30:27	18.09.2026, 23:59:59	11.09.2026, 22:30:39	18.09.2026, 23:59:59	11.09.2026, 22:30:58	-	RETURNED	Details Rent Return Set Overdue Remove from Cart Delete
2	2 / Piotr2 Owczarek2	Test Drive Unlimited	14.09.2026, 02:42:51	21.09.2026, 23:59:59	14.09.2026, 02:51:00	21.09.2026, 23:59:59	-	-	ACTIVE	Details Rent Return Set Overdue Remove from Cart Delete
3	2 / Piotr2 Owczarek2	The Odessa Chronicles	14.09.2026, 02:49:55	21.09.2026, 23:59:59	14.09.2026, 02:51:01	21.09.2026, 23:59:59	-	14.09.2026, 02:51:02	OVERDUE	Details Rent Return Set Overdue Remove from Cart Delete

- Fees - /admin-panel/fees

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Fees Report

Search fee by user name

Clear filters

#	(ID) User	Total Fees	Fees PENDING	Fees PAID	Actions
1	(1) Piotr Owczarek	0	0	0	View Details
2	(2) Piotr2 Owczarek2	1	1	0	View Details
3	(3) Piotr3 Owczarek3	0	0	0	View Details

- Fee Items - /admin-panel/feeltems

ADMIN Panel

User: Piotr Owczarek (piotr2785.owca@gmail.com)

Navigation

Dashboard
Books
Physical Books
Categories
Authors
Users
Loans
Loan Items
Fees
Fee Items

Fee Items Dashboar

Search fee by user, fee ID, loan ID or book title

Clear filters

#	ID / User	Book / Authors / Inventory Code	Created At	Amount	Paid At	Status	All	Actions
1	2 / Piotr2 Owczarek2	The Odessa Chronicles / 0 / 3	14.09.2026, 02:51:02	5.00 zł	-	PENDING		Details Mark as Paid Delete